11 Implementing authorization at the method level

This chapter covers

- Method security in Spring applications
- Preauthorization of methods based on authorities, roles, and permissions
- Postauthorization of methods based on authorities, roles, and permissions

Up to now, we've discussed various ways of configuring authentication. We started with the most straightforward approach, HTTP Basic, in chapter 2, and then I showed you how to set form login in chapter 6. However, in terms of authorization, we only discussed configuration at the endpoint level. Suppose your app is not a web application—can't you use Spring Security for authentication and authorization as well? Spring Security is a good fit for scenarios where your app isn't used via HTTP endpoints. In this chapter, you'll learn how to configure authorization at the method level. We'll use this approach to configure authorization in both web and non-web applications, and we'll call it *method security* (figure 11.1).

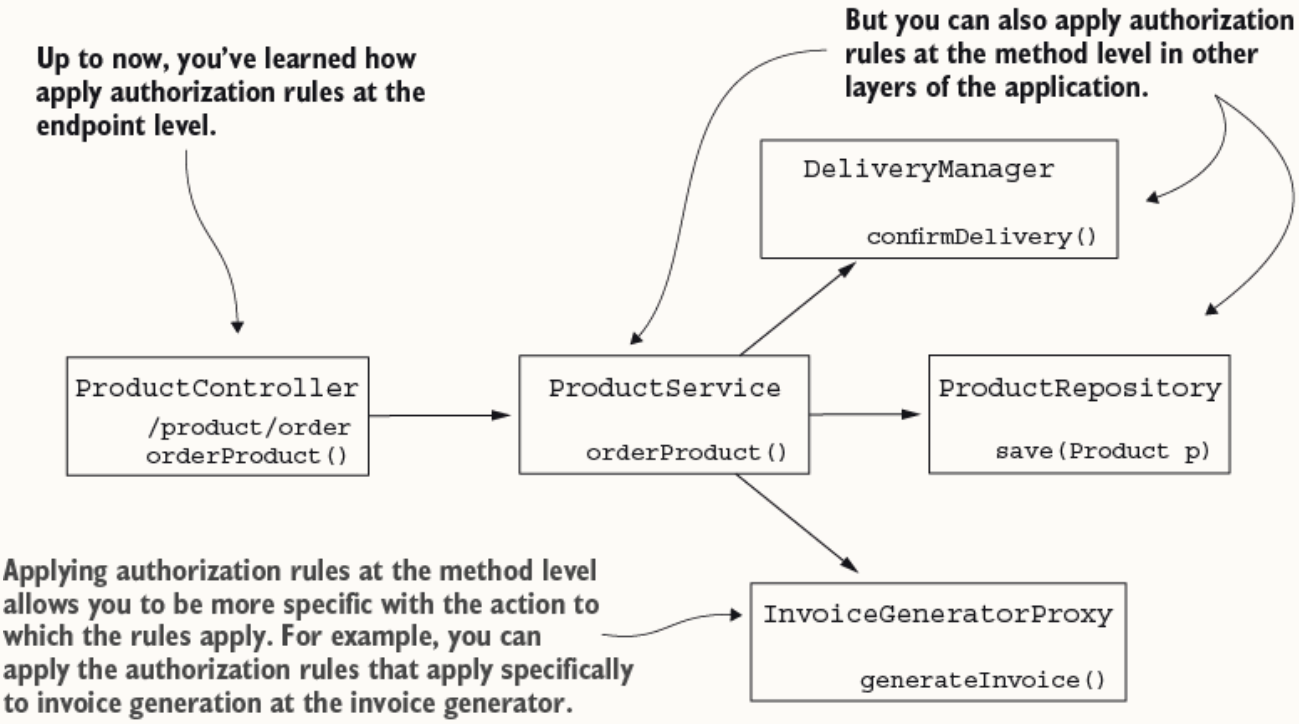


Figure 11.1 Method security allows you to be more granular and apply authorization rules at any specifically chosen level of your application.

For non-web applications, method security allows us to implement authorization rules even if we don't have endpoints. In web applications, this approach gives us the flexibility to apply authorization rules to different app layers, not just at the endpoint level. Let's dive into the chapter and learn how to apply authorization at the method level with method security.

11.1 Enabling method security

This section will show how to enable authorization at the method level and the different options that Spring Security offers for applying various authorization rules. This approach provides you with greater flexibility in applying authorization. It's an essential skill that allows you to solve situations in which authorization cannot be configured just at the endpoint level.

By default, method security is disabled, so if you want to use this functionality, you first need to enable it. Furthermore, method security offers multiple approaches for applying authorization. We discuss these approaches and then implement them in examples in the following sections of this chapter and in chapter 12. Briefly, you can do two main things with global method security:

- *Call authorization*—Decides whether someone can call a method according to some implemented privilege rules (preauthorization) or if someone can access what the method returns after the method executes (postauthorization).
- *Filtering*—Decides what a method can receive through its parameters (prefiltering) and what the caller can receive back from the method after the method executes (postfiltering). We'll discuss and implement filtering in chapter 12.

11.1.1 Understanding call authorization

One of the approaches for configuring authorization rules used with method security is *call authorization*. The call authorization approach refers to applying authorization rules that decide if a method can be called or that allow the method to be called and then decide if the caller can access the value returned by the method. Often, we need to decide if someone can access a piece of logic depending on either the provided parameters or its result. So let's discuss call authorization and then apply it to some examples.

How does method security work? What's the mechanism behind applying the authorization rules? When we enable method security in our application, we actually enable a Spring aspect. This aspect intercepts the calls to the method for which we apply autho-

rization rules, and based on these authorization rules, it decides whether to forward the call to the intercepted method (figure 11.2).

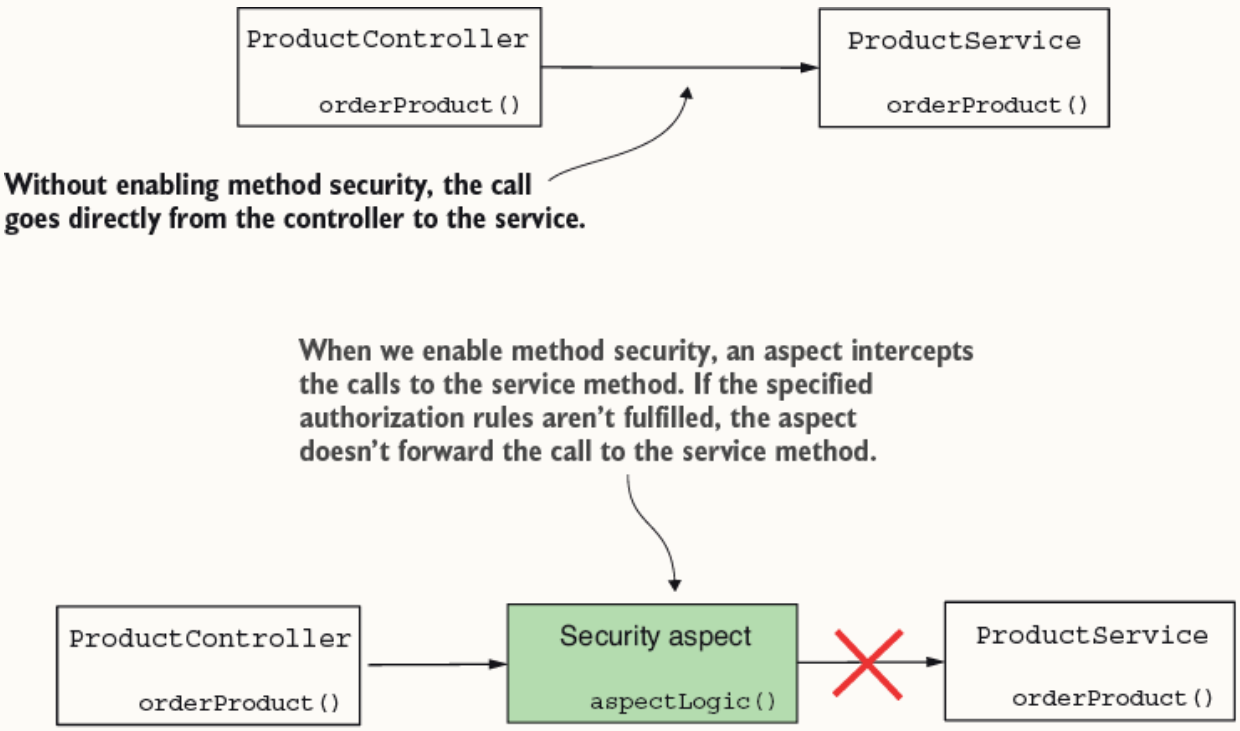


Figure 11.2 When we enable global method security, an aspect intercepts the call to the protected method. If the given authorization rules aren't respected, the aspect doesn't delegate the call to the protected method.

Plenty of implementations in Spring framework rely on aspect-oriented programming (AOP). Method security is just one of the many components in Spring applications relying on aspects. If you need a refresher on aspects and AOP, I recommend you read chapter 6 of *Spring Start Here* (Manning, 2021), another book I wrote. Briefly, we classify the call authorization as

- *Preauthorization*—The framework checks the authorization rules before the method call.
- *Postauthorization*—The framework checks the authorization rules after the method executes.

Let's take both approaches, describe them, and implement them with some examples.

Using preauthorization to secure access to methods

Say we have a method `findDocumentsByUser(String username)` that returns to the caller documents for a specific user. The caller provides through the method's parameters the user's name for which the method retrieves the documents. Assume you need to ensure that the authenticated user can only obtain their own documents. Can we apply a rule to this method so that only the method calls that receive the username of the au-

thenticated user as a parameter are allowed? Yes! This is something we do with preauthorization.

When we apply authorization rules that completely forbid anyone to call a method in specific situations, we call this *preauthorization* (figure 11.3). This approach implies that the framework verifies the authorization conditions before executing the method. If the caller doesn't have the permissions according to the authorization rules that we define, the framework doesn't delegate the call to the method. Instead, the framework throws an exception named `AccessDeniedException`. This is by far the most frequently used approach to global method security.

Figure 11.3 With preauthorization, the authorization rules are verified before delegating the method call further. The framework won't delegate the call if the authorization rules aren't respected, and instead, it will throw an exception to the method caller.

Usually, we don't want a functionality to be executed at all if some conditions aren't met. You can apply conditions based on the authenticated user, and you can also refer to the values the method received through its parameters.

Using postauthorization to secure a method call

When we apply authorization rules that allow someone to call a method but not necessarily to obtain the result returned, we're using *postauthorization* (figure 11.4). With postauthorization, Spring Security checks the authorization rules after the method executes. You can use this kind of authorization to restrict access to the method return in certain conditions. Because postauthorization happens after method execution, you can apply the authorization rules on the result returned.

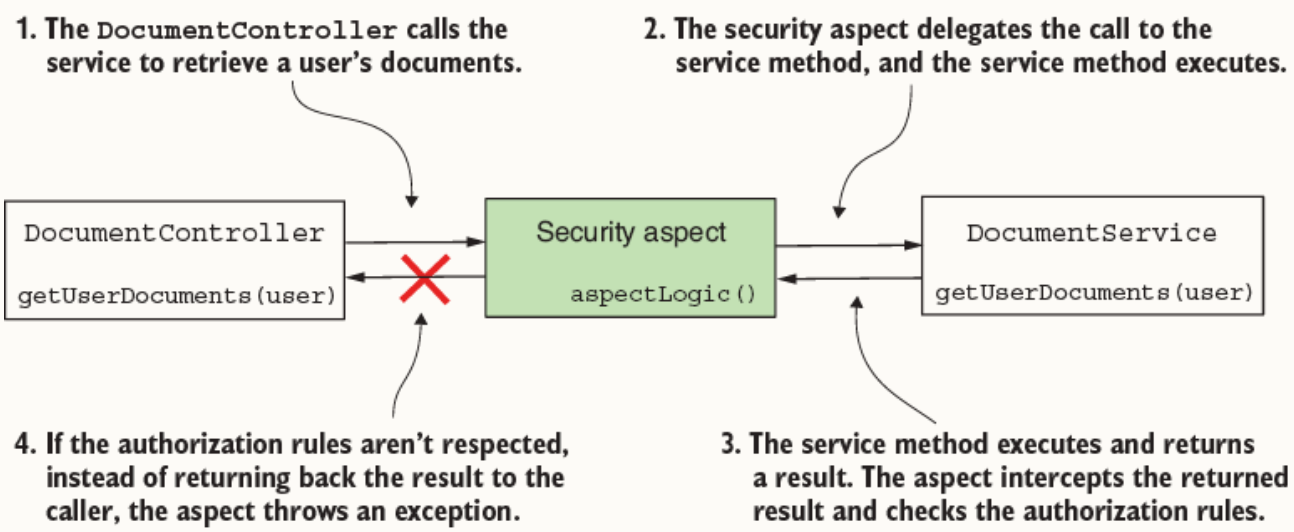


Figure 11.4 With postauthorization, the aspect delegates the call to the protected method. After the protected method finishes execution, the aspect checks the authorization rules. If the rules aren't respected, instead of returning the result to the caller, the aspect throws an exception.

Usually, we use postauthorization to apply authorization rules based on what the method returns after execution. But be careful with postauthorization! If the method mutates something during its execution, the change happens regardless of whether authorization is successful.

NOTE Even with the `@Transactional` annotation, a change isn't rolled back if postauthorization fails. The exception thrown by the postauthorization functionality happens after the transaction manager commits the transaction.

11.1.2 Enabling method security in your project

In this section, we'll work on a project to apply the preauthorization and postauthorization features offered by method security. Method security isn't enabled by default in a Spring Security project. To use it, you need to first enable it. However, enabling this functionality is straightforward. You do this by simply using the `@EnableMethodSecurity` annotation on the configuration class.

I created a new project for this example, ssia-ch11-ex1. For this project, I wrote a `ProjectConfig` configuration class, as presented in listing 11.1. On the configuration class, we add the `@EnableMethodSecurity` annotation. Method security offers us the following three approaches to define the authorization rules that we discuss in this chapter:

- The pre-/postauthorization annotations (enabled by default)
- The JSR 250 annotation, `@RolesAllowed`
- The `@Secured` annotation

Because in almost all cases, pre-/postauthorization annotations are the only approach used, we discuss it in this chapter. This approach is pre-enabled once you add the `@EnableMethodSecurity` annotation. We present a short overview of the other two options at the end of this chapter.

Listing 11.1 Enabling method security

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {
}
```

You can use global method security with any authentication approach, from HTTP Basic authentication to OAuth 2 (which you'll learn in the third part of this book). To keep it simple and allow you to focus on new details, we provide method security with HTTP Basic authentication. For this reason, the pom.xml file for the projects in this chapter only needs the web and Spring Security dependencies, as the next code snippet presents:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

NOTE In the previous Spring Security versions, we used the `@EnableGlobalMethodSecurity` annotation, and the pre- and postauthorization weren't enabled by default. If you need to work with method authorization and a previous Spring Security version (older than 6), you may find chapter 16 from the first edition of *Spring Security in Action* useful.

11.2 Applying preauthorization rules

In this section, we implement an preauthorization example. For our example, we continue with the project `ssia-ch11-ex1` started in section 11.1. As discussed in section 11.1, preauthorization implies defining authorization rules that Spring Security applies before

calling a specific method. If the rules aren't respected, the framework doesn't call the method.

The application we implement in this section has a simple scenario. It exposes an end-point, /hello, which returns the string `"Hello"`, followed by a name. To obtain the name, the controller calls a service method (figure 11.5). This method applies a preauthorization rule to verify the user has write authority.

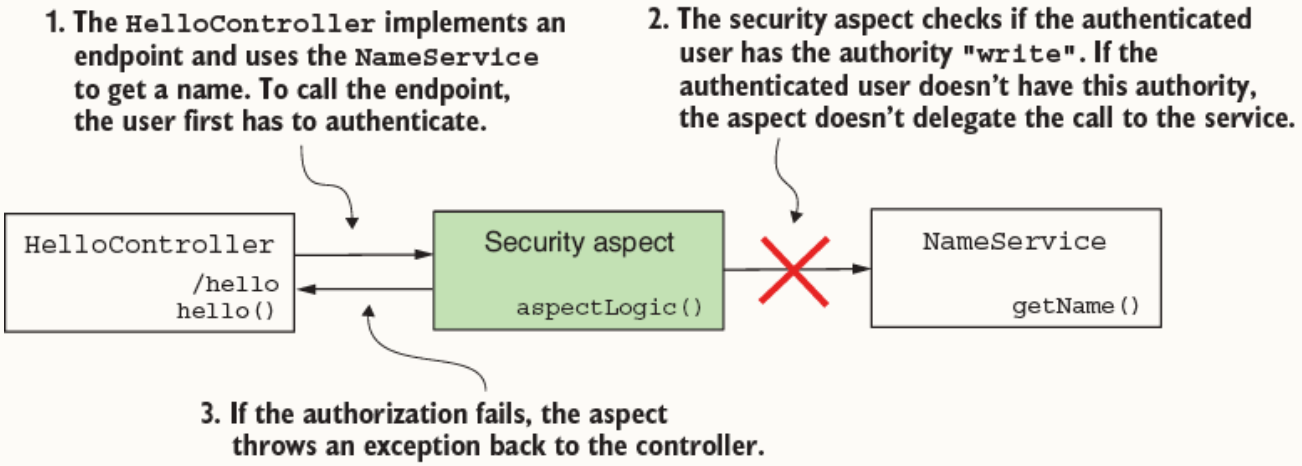


Figure 11.5 To call the `getName()` method of `NameService`, the authenticated user needs to have write authority. If the user doesn't have this authority, the framework won't allow the call and will throw an exception.

I added a `UserDetailsService` and a `PasswordEncoder` to make sure I have some users to authenticate. To validate our solution, we need two users: one with write authority and another without it. We prove that the first user can successfully call the endpoint, while for the second user, the app throws an authorization exception when trying to call the method. The following listing shows the complete definition of the configuration class, which defines the `UserDetailsService` and the `PasswordEncoder`.

Listing 11.2 The configuration class for `UserDetailsService` and `PasswordEncoder`

```
@Configuration
@EnableMethodSecurity                    ①
public class ProjectConfig {

 @Bean                                   ②
 public UserDetailsService userDetailsService() {
  var service = new InMemoryUserDetailsManager();

  var u1 = User.withUsername("natalie")
         .password("12345")
         .authorities("read")
         .build();
```

```
    var u2 = User.withUsername("emma")
            .password("12345")
            .authorities("write")
            .build();

    service.createUser(u1);
    service.createUser(u2);

    return service;
  }

  @Bean                          ③
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① Enables method security for pre-/postauthorization

② Adds a UserDetailsService to the Spring context with two users for testing

③ Adds a PasswordEncoder to the Spring context

To define the authorization rule for this method, we use the `@PreAuthorize` annotation. The `@PreAuthorize` annotation receives as a value a Spring Expression Language (SpEL) expression that describes the authorization rule. In this example, we apply a simple rule.

You can define restrictions for users based on their authorities using the `hasAuthority()` method. You learned about the `hasAuthority()` method in chapter 7, where we discussed applying authorization at the endpoint level. The following listing defines the service class, which provides the value for the name.

Listing 11.3 The service class defining the preauthorization rule on the method

```
@Service
public class NameService {

  @PreAuthorize("hasAuthority('write')")      ①
  public String getName() {
    return "Fantastico";
  }
}
```

① Defines the authorization rule. Only users having write authority can call the method.

We define the controller class in the following listing. It uses `NameService` as a dependency.

Listing 11.4 The controller class implementing the endpoint and using the service

```
@RestController
public class HelloController {

  private final NameService nameService;        ①

  // omitted constructor

  @GetMapping("/hello")
  public String hello() {
    return "Hello, " + nameService.getName();    ②
  }
}
```

① Injects the service from the context

② Calls the method for which we apply the preauthorization rules

You can now start the application and test its behavior. We expect only user Emma to be authorized to call the endpoint because she has written authorization. The next code snippet presents the calls for the endpoint with our two users, Emma and Natalie. To call the /hello endpoint and authenticate with user Emma, use this cURL command:

```
curl -u emma:12345 http://localhost:8080/hello
```

The response body is

```
Hello, Fantastico
```

To call the /hello endpoint and authenticate with user Natalie, use this cURL command:

```
curl -u natalie:12345 http://localhost:8080/hello
```

The response body is

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

Similarly, you can use any other expression we discussed in chapter 7 for endpoint authentication. Here's a short recap of them:

- `hasAnyAuthority()` —Specifies multiple authorities. The user must have at least one of these authorities to call the method.
- `hasRole()` —Specifies a role a user must have to call the method.
- `hasAnyRole()` —Specifies multiple roles. The user must have at least one of them to call the method.

Let's extend our example to prove how you can use the values of the method parameters to define the authorization rules (figure 11.6). You find this example in the project named ssia-ch11-ex2.

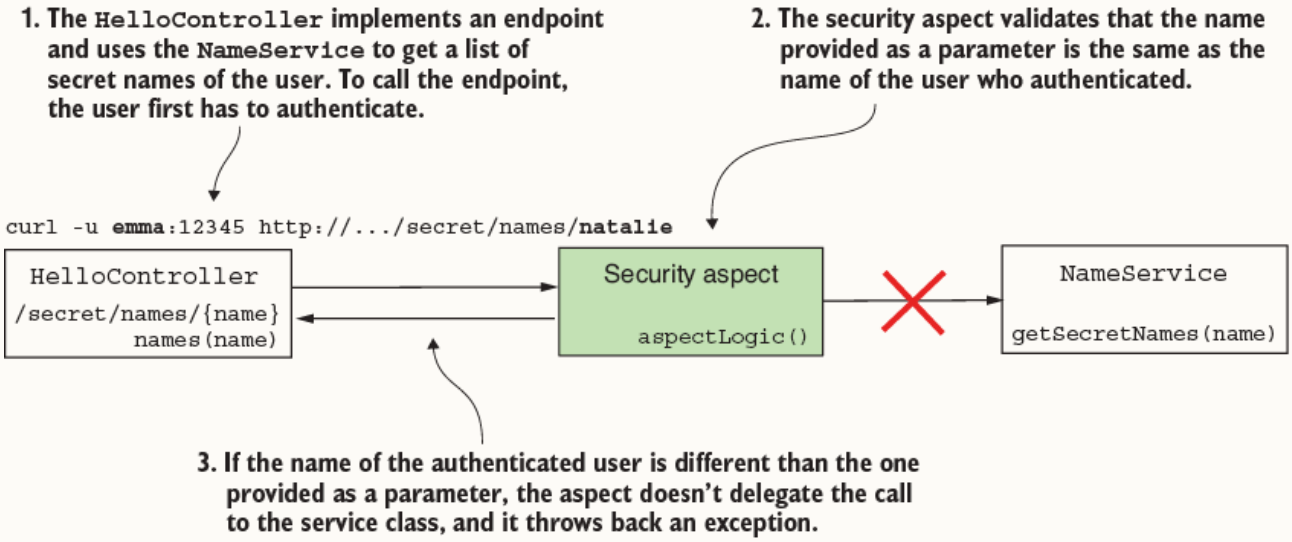


Figure 11.6 When implementing preauthorization, we can use the values of the method parameters in the authorization rules. In our example, only the authenticated user can retrieve information about their secret names.

For this project, I defined the same `ProjectConfig` class as in our first example so that we can continue working with our two users, Emma and Natalie. The endpoint now takes a value through a path variable and calls a service class to obtain the "secret names" for a given username. Of course, in this case, the secret names are just an invention of mine

referring to a characteristic of the user, which is something that not everyone can see. I define the controller class as presented in the next listing.

Listing 11.5 The controller class defining an endpoint for testing

```
@RestController
public class HelloController {

  private final NameService nameService;                ①

  // omitted constructor

  @GetMapping("/secret/names/{name}")                   ②
  public List<String> names(@PathVariable String name) {
     return nameService.getSecretNames(name);           ③
  }
}
```

① From the context, injects an instance of the service class that defines the protected method

② Defines an endpoint that takes a value from a path variable

③ Calls the protected method to obtain the secret names of the users

Now let's take a look at how to implement the `NameService` class in listing 11.6. The expression we use for authorization now is `#name == authentication.principal .username`. In this expression, we use `#name` to refer to the value of the `getSecretNames()` method parameter called `name`, and we have access directly to the authentication object that we can use to refer to the currently authenticated user. The expression we use indicates that the method can be called only if the authenticated user's username is the same as the value sent through the method's parameter. In other words, a user can only retrieve its own secret names.

Listing 11.6 The `NameService` class defining the protected method

```
@Service
public class NameService {

  private Map<String, List<String>> secretNames =
    Map.of(
```

```
    "natalie", List.of("Energico", "Perfecto"),
    "emma", List.of("Fantastico"));

  @PreAuthorize                                ①
   ("#name == authentication.principal.username")
  public List<String> getSecretNames(String name) {
   return secretNames.get(name);
  }
}
```

① Uses #name to represent the value of the method parameters in the authorization expression

We start the application and test it to prove it works as desired. The next code snippet shows the application behavior when calling the endpoint, providing the value of the path variable equal to the name of the user:

```
curl -u emma:12345 http://localhost:8080/secret/names/emma
```

The response body is

```
["Fantastico"]
```

When authenticating with the user Emma, we try to get Natalie's secret names. The call doesn't work:

```
curl -u emma:12345 http://localhost:8080/secret/names/natalie
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
 "message":"Forbidden",
 "path":"/secret/names/natalie"
}
```

The user Natalie can, however, obtain her own secret names. The next code snippet proves this:

```
curl -u natalie:12345 http://localhost:8080/secret/names/natalie
```

The response body is

```
["Energico","Perfecto"]
```

NOTE Remember, you can apply method security to any layer of your application. In the examples presented in this chapter, you find the authorization rules applied for methods of the service classes. However, you can apply authorization rules with method security to any part of your application: controllers, repositories, managers, proxies, and so on.

11.3 Applying postauthorization rules

Say you want to allow a call to a method, but in certain circumstances, you want to ensure the caller doesn't receive the returned value. When we want to apply an authorization rule that is verified after the call of a method, we use postauthorization. It may sound a little bit awkward at the beginning: Why would someone be able to execute the code but not get the result? Well, it's not about the method itself, but imagine this method retrieves some data from a data source, say, a web service or a database. The conditions you need to add for authorization depend on the received data. So you allow the method to execute, but you validate what it returns, and if it doesn't meet the criteria, you don't let the caller access the return value.

To apply postauthorization rules with Spring Security, we use the `@PostAuthorize` annotation, which is similar to `@PreAuthorize`, discussed in section 11.2. The annotation receives the SpEL as a value, defining an authorization rule. We continue with an example showing how to use the `@PostAuthorize` annotation and define postauthorization rules for a method (figure 11.7).

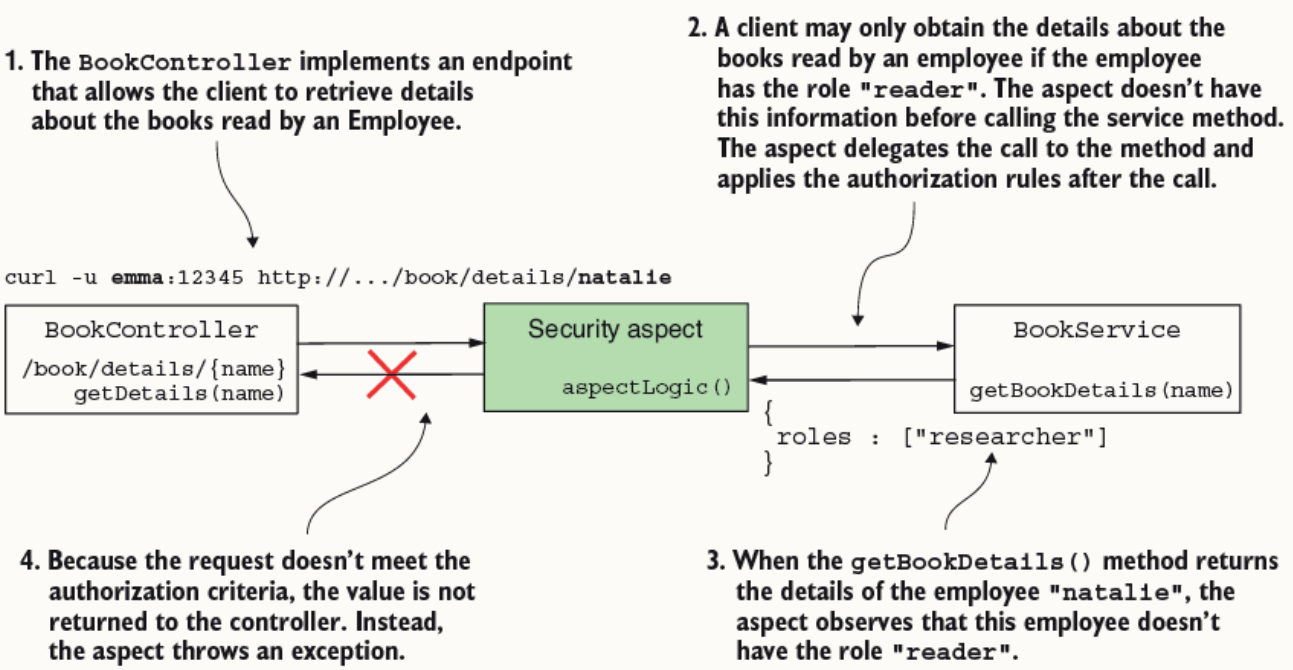


Figure 11.7 With postauthorization, we don't protect the method from being called, but we protect the returned value from being exposed if the defined authorization rules aren't respected.

The scenario for our example, for which I created a project named ssia-ch11-ex3, defines an object `Employee`. Our `Employee` has a name, a list of books, and a list of authorities. We associate each `Employee` to a user of the application. To stay consistent with the other examples in this chapter, we define the same users, Emma and Natalie. We want to make sure that the method caller gets the employee details only if the employee has read authority. Because we don't know the authorities associated with the employee record until we retrieve it, we need to apply the authorization rules after the method execution. For this reason, we use the `@PostAuthorize` annotation.

The configuration class is like the one we used in the previous examples. But for your convenience, I repeat it in the next listing.

Listing 11.7 Enabling method security and defining users

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

 @Bean
 public UserDetailsService userDetailsService() {
  var service = new InMemoryUserDetailsManager();

  var u1 = User.withUsername(„natalie")
            .password("12345")
            .authorities("read")
            .build();
```

```
    var u2 = User.withUsername("emma")
              .password("12345")
              .authorities("write")
              .build();

    service.createUser(u1);
    service.createUser(u2);

    return service;
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

We also need to declare a class to represent the `Employee` object with its name, book list, and roles list. The following listing defines the `Employee` class.

Listing 11.8 The definition of the `Employee` class

```
public class Employee {

  private String name;
  private List<String> books;
  private List<String> roles;

  // Omitted constructor, getters, and setters
}
```

We probably get our employee details from a database. To keep our example shorter, I use a `Map` with a couple of records that we consider as our data source. In listing 11.9, you find the definition of the `BookService` class. The `BookService` class also contains the method for which we apply the authorization rules. Observe that the expression we use with the `@PostAuthorize` annotation refers to the value returned by the method `returnObject`. The postauthorization expression can use the value returned by the method, which is available after the method executes.

Listing 11.9 The `BookService` class defining the authorized method

```
@Service
public class BookService {

  private Map<String, Employee> records =
    Map.of("emma",
          new Employee("Emma Thompson",
             List.of("Karamazov Brothers"),
             List.of("accountant", "reader")),
          "natalie",
          new Employee("Natalie Parker",
             List.of("Beautiful Paris"),
             List.of("researcher"))
        );
  @PostAuthorize("returnObject.roles.contains('reader')")      ①
  public Employee getBookDetails(String name) {
     return records.get(name);
  }
}
```

① Defines the expression for postauthorization

Let's also write a controller and implement an endpoint to call the method for which we applied the authorization rule. The following listing presents this controller class.

Listing 11.10 The controller class implementing the endpoint

```
@RestController
public class BookController {

  private final BookService bookService;

  // omitted constructor

  @GetMapping("/book/details/{name}")
  public Employee getDetails(@PathVariable String name) {
    return bookService.getBookDetails(name);
  }
}
```

You can now start the application and call the endpoint to observe the app's behavior. In the next code snippets, you find examples of calling the endpoint. Any of the users can access the details of Emma because the returned list of roles contains the string

"reader", but no user can obtain the details for Natalie. Calling the endpoint to get the details for Emma and authenticating with user Emma, we implement the following command:

```
curl -u emma:12345 http://localhost:8080/book/details/emma
```

The response body is

```
{
 "name":"Emma Thompson",
 "books":["Karamazov Brothers"],
 "roles":["accountant","reader"]
}
```

Calling the endpoint to get the details for Emma and authenticating with user Natalie, we use

```
curl -u natalie:12345 http://localhost:8080/book/details/emma
```

The response body is

```
{
 "name":"Emma Thompson",
 "books":["Karamazov Brothers"],
 "roles":["accountant","reader"]
}
```

Calling the endpoint to get the details for Natalie and authenticating with user Emma, we use

```
curl -u emma:12345 http://localhost:8080/book/details/natalie
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
```

```
  "message":"Forbidden",
  "path":"/book/details/natalie"
}
```

Calling the endpoint to get the details for Natalie and authenticating with user Natalie, we use this command:

```
curl -u natalie:12345 http://localhost:8080/book/details/natalie
```

The response body is

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/book/details/natalie"
}
```

NOTE You can use both `@PreAuthorize` and `@PostAuthorize` for the same method if your requirements need to have both preauthorization and postauthorization.

11.4 Implementing permissions for methods

Up to now, you learned how to define rules with simple expressions for preauthorization and postauthorization. Now let's assume the authorization logic is more complex, and you cannot write it in one line. It's definitely not comfortable to write huge SpEL expressions. I never recommend using long SpEL expressions in any situation, regardless of whether it's an authorization rule or not. It simply creates hard-to-read code, and this affects the app's maintainability. When you need to implement complex authorization rules instead of writing long SpEL expressions, take the logic out in a separate class. Spring Security provides the concept of *permission*, which makes it easy to write the authorization rules in a separate class so that your application is easier to read and understand.

In this section, we apply authorization rules using permissions in a project. I named this project `ssia-ch11-ex4`. In this scenario, you have an application managing documents. Every document has an owner, which is the user who created the document. To get the details of an existing document, a user either must be an admin or they must be the

owner of the document. We implement a permission evaluator to solve this requirement. The following listing defines the `document`, which is only a plain Java object.

Listing 11.11 The `Document` class

```
public class Document {

  private String owner;

  // Omitted constructor, getters, and setters
}
```

To mock the database and make our example shorter for your comfort, I created a repository class that manages a few document instances in a `Map`. This class is presented in the next listing.

Listing 11.12 The `DocumentRepository` class managing a few `Document` instances

```
@Repository
public class DocumentRepository {

  private Map<String, Document> documents =          ①
    Map.of(«abc123», new Document(«natalie»),
          «qwe123», new Document(«natalie»),
          «asd555», new Document(«emma»));

  public Document findDocument(String code) {
    return documents.get(code);                      ②
  }
}
```

① Identifies each document by a unique code and names the owner

② Obtains a document by using its unique identification code

A service class defines a method that uses the repository to obtain a document by its code. The method in the service class is the one for which we apply the authorization rules. The logic of the class is simple. It defines a method that returns the `Document` by its unique code. We annotate this method with `@PostAuthorize` and use a `hasPermission()` SpEL expression. This method allows us to refer to an external authorization expression that we implement further in this example. Meanwhile, observe that the param-

eters we provide to the `hasPermission()` method are the `returnObject`, which represents the value returned by the method, and the name of the role for which we allow access, which is `'ROLE_admin'`. The definition of this class is presented in the following listing.

Listing 11.13 The `DocumentService` class implementing the protected method

```
@Service
public class DocumentService {

  private final DocumentRepository documentRepository;

  // omitted constructor

  @PostAuthorize                                    ①
  ("hasPermission(returnObject, 'ROLE_admin')")
  public Document getDocument(String code) {
    return documentRepository.findDocument(code);
  }
}
```

① Uses the hasPermission() expression to refer to an authorization expression

It's our duty to implement the permission logic. And we do this by writing an object that implements the `PermissionEvaluator` contract. The `PermissionEvaluator` contract provides two ways to implement the permission logic:

- *By object and permission*—Used in the current example, it assumes the permission evaluator receives two objects: one that's subject to the authorization rule and one that offers extra details needed for implementing the permission logic.
- *By object ID, object type, and permission*—Assumes the permission evaluator receives an object ID, which it can use to retrieve the needed object. It also receives a type of object, which can be used if the same permission evaluator applies to multiple object types, and it needs an object offering extra details for evaluating the permission.

In the next listing, you find the `PermissionEvaluator` contract with two methods.

Listing 11.14 The `PermissionEvaluator` contract definition

```
public interface PermissionEvaluator {
```

```
    boolean hasPermission(
            Authentication a,
            Object subject,
            Object permission);

    boolean hasPermission(
            Authentication a,
            Serializable id,
            String type,
            Object permission);
}
```

For the current example, it's enough to use the first method. We already have the subject, which in our case, is the value returned by the method. We also send the role name `'ROLE_admin'`, which, as defined by the example's scenario, can access any document. Of course, in our example, we could have directly used the name of the role in the permission evaluator class and avoided sending it as a value of the `hasPermission()` object. Here, we only do the former for the sake of the example. In a real-world scenario, which might be more complex, you have multiple methods, and details needed in the authorization process might differ between each of them. For this reason, you have a parameter that you can send the needed details for use in the authorization logic from the method level.

For your awareness and to avoid confusion, I'd also like to mention that you don't have to pass the `Authentication` object. Spring Security automatically provides this parameter value when calling the `hasPermission()` method. The framework knows the value of the authentication instance because it is already in the `SecurityContext`. In the next listing, you find the `DocumentsPermissionEvaluator` class, which in our example implements the `PermissionEvaluator` contract to define the custom authorization rule.

Listing 11.15 Implementing the authorization rule

```
@Component
public class DocumentsPermissionEvaluator
 implements PermissionEvaluator {              ①

 @Override
 public boolean hasPermission(
  Authentication authentication,
  Object target,
  Object permission) {
```

```
    Document document = (Document) target;        ②
    String p = (String) permission;               ③

    boolean admin =                               ④
      authentication.getAuthorities()
        .stream()
        .anyMatch(a -> a.getAuthority().equals(p));

    return admin ||                               ⑤
      document.getOwner()
        .equals(authentication.getName());

  }

  @Override
  public boolean hasPermission(Authentication authentication,
                               Serializable targetId,
                               String targetType,
                               Object permission) {
    return false;                                 ⑥
  }
}
```

① Implements the PermissionEvaluator contract

② Casts the target object to Document

③ The permission object in our case is the role name, so we cast it to a String.

④ Checks if the authentication user has the role we got as a parameter

⑤ If admin or the authenticated user is the owner of the document, grants the permission

⑥ We don't need to implement the second method because we don't use it.

To make Spring Security aware of our new `PermissionEvaluator` implementation, we have to define a `MethodSecurityExpressionHandler` bean in the configuration class. The following listing presents how to define a `MethodSecurityExpressionHandler` to make the custom `PermissionEvaluator` known .

Listing 11.16 Configuring the `PermissionEvaluator` in the configuration class

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

  private final DocumentsPermissionEvaluator evaluator;

  // omitted constructor

  @Bean                                          ①
  protected MethodSecurityExpressionHandler createExpressionHandler() {
    var expressionHandler =                      ②
        new DefaultMethodSecurityExpressionHandler();

    expressionHandler.setPermissionEvaluator(
        evaluator);                              ③

    return expressionHandler;                    ④
  }

  // Omitted definition of the UserDetailsService and PasswordEncoder beans
}
```

① Makes the returned MethodSecurityExpressionHandler object a bean in the Spring context

② Creates a default security expression handler to set up the custom permission evaluator

③ Sets up the custom permission evaluator

④ Returns the custom expression handler to be added to the Spring context

NOTE We use here an implementation for `MethodSecurityExpressionHandler` named `DefaultMethodSecurityExpressionHandler` that Spring Security provides. You could also implement a custom `MethodSecurityExpressionHandler` to define custom SpEL expressions you use to apply the authorization rules. You rarely need to do this in a real-world scenario, and for this reason, we won't implement such a custom object in our examples. I just wanted to make you aware that this is possible.

I separate the definition of the `UserDetailsService` and `PasswordEncoder` to let you focus only on the new code. In listing 11.17, you can find the rest of the configuration class. The only important thing to notice about the users is their roles. User Natalie is an admin

and can access any document. User Emma is a manager and can only access her own documents.

Listing 11.17 The full definition of the configuration class

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

 private final DocumentsPermissionEvaluator evaluator;

 // Omitted constructor

 @Override
 protected MethodSecurityExpressionHandler createExpressionHandler() {
  var expressionHandler =
     new DefaultMethodSecurityExpressionHandler();

  expressionHandler.setPermissionEvaluator(evaluator);

  return expressionHandler;
 }

 @Bean
 public UserDetailsService userDetailsService() {
  var service = new InMemoryUserDetailsManager();

  var u1 = User.withUsername("natalie")
         .password("12345")
         .roles("admin")
         .build();

   var u2 = User.withUsername("emma")
          .password("12345")
          .roles("manager")
          .build();

   service.createUser(u1);
   service.createUser(u2);

   return service;
 }

 @Bean
 public PasswordEncoder passwordEncoder() {
```

```
    return NoOpPasswordEncoder.getInstance();
  }
}
```

To test the application, we define an endpoint. The following listing presents this definition.

Listing 11.18 Defining the controller class and implementing an endpoint

```
@RestController
public class DocumentController {

  private final DocumentService documentService;

  // Omitted constructor

  @GetMapping("/documents/{code}")
  public Document getDetails(@PathVariable String code) {
    return documentService.getDocument(code);
  }
}
```

Let's run the application and call the endpoint to observe its behavior. User Natalie can access the documents regardless of their owner. User Emma can only access the documents she owns. Calling the endpoint for a document that belongs to Natalie and authenticating with the user `"natalie"`, we use this command:

```
curl -u natalie:12345 http://localhost:8080/documents/abc123
```

The response body is

```
{
  "owner":"natalie"
}
```

Calling the endpoint for a document that belongs to Emma and authenticating with the user `"natalie"`, we use

```
curl -u natalie:12345 http://localhost:8080/documents/asd555
```

The response body is

```
{
 "owner":"emma"
}
```

Calling the endpoint for a document that belongs to Emma and authenticating with the user `"emma"`, we use

```
curl -u emma:12345 http://localhost:8080/documents/asd555
```

The response body is

```
{
 "owner":"emma"
}
```

Calling the endpoint for a document that belongs to Natalie and authenticating with the user `"emma"`, we use

```
curl -u emma:12345 http://localhost:8080/documents/abc123
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
 "message":"Forbidden",
 "path":"/documents/abc123"
}
```

In a similar manner, you can use the second `PermissionEvaluator` method to write your authorization expression. The second method refers to using an identifier and subject type instead of the object itself. For example, say that we want to change the current example to apply the authorization rules before the method is executed, using `@PreAuthorize`. In this case, we don't have the returned object yet. But instead of having the object itself, we have the document's code, which is its unique identifier. The

next listing shows you how to change the permission evaluator class to implement this scenario. I separated the examples in a project named ssia-ch11-ex5 , which you can run individually.

Listing 11.19 Changes in the DocumentsPermissionEvaluator class

```
@Component
public class DocumentsPermissionEvaluator
 implements PermissionEvaluator {

 private final DocumentRepository documentRepository;

 // Omitted constructor

 @Override
 public boolean hasPermission(Authentication authentication,
                              Object target,
                              Object permission) {
  return false;                                    ①
 }

 @Override
 public boolean hasPermission(Authentication authentication,
                              Serializable targetId,
                              String targetType,
                              Object permission) {

  String code = targetId.toString();                    ②
  Document document = documentRepository.findDocument(code);

  String p = (String) permission;

  boolean admin =
        authentication.getAuthorities()
          .stream()
          .anyMatch(a -> a.getAuthority().equals(p));

   return admin ||                                  ③
     document.getOwner().equals(
      authentication.getName());
 }
}
```

① No longer defines the authorization rules through the first method

② Instead of having the object, we have its ID, and we get the object using the ID.

③ If the user is an admin or the owner of the document, the user can access the document.

Of course, we also need to use the proper call to the permission evaluator with the `@PreAuthorize` annotation. In the following listing, you find the change I made in the `DocumentService` class to apply the authorization rules with the new method.

Listing 11.20 The `DocumentService` class

```
@Service
public class DocumentService {

 private final DocumentRepository documentRepository;

 // Omitted contructor

 @PreAuthorize                                    ①
  ("hasPermission(#code, 'document', 'ROLE_admin')")
 public Document getDocument(String code) {
  return documentRepository.findDocument(code);
 }
}
```

① Applies the preauthorization rules by using the second method of the permission evaluator

You can rerun the application and check the behavior of the endpoint. You should see the same result as in the case where we used the first method of the permission evaluator to implement the authorization rules. The user Natalie is an admin and can access details of any document, while the user Emma can only access the documents she owns. Calling the endpoint for a document that belongs to Natalie and authenticating with the user `"natalie"`, we issue

```
curl -u natalie:12345 http://localhost:8080/documents/abc123
```

The response body is

```
{
 "owner":"natalie"
}
```

Calling the endpoint for a document that belongs to Emma and authenticating with the user `"natalie"`, we issue

```
curl -u natalie:12345 http://localhost:8080/documents/asd555
```

The response body is

```
{
 "owner":"emma"
}
```

Calling the endpoint for a document that belongs to Emma and authenticating with the user `"emma"`, we issue

```
curl -u emma:12345 http://localhost:8080/documents/asd555
```

The response body is

```
{
 "owner":"emma"
}
```

Calling the endpoint for a document that belongs to Natalie and authenticating with the user `"emma"`, we issue

```
curl -u emma:12345 http://localhost:8080/documents/abc123
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
```

```
  "message":"Forbidden",
  "path":"/documents/abc123"
}
```

Using the @Secured and @RolesAllowed annotations

Throughout this chapter, we discussed applying authorization rules with global method security. We started by learning that this functionality is disabled by default and that you can enable it using the `@EnableMethodSecurity` annotation over the configuration class. Moreover, when using pre- and postauthorization, you don't need to specify a certain way to apply the authorization rules using an attribute of the `@EnableMethodSecurity` annotation. We used the annotation like this:

```
@EnableMethodSecurity
```

The `@EnableMethodSecurity` annotation offers two attributes that you can use to enable different annotations. You use the `jsr250Enabled` attribute to enable the `@RolesAllowed` annotation and the `securedEnabled` attribute to enable the `@Secured` annotation. Using these two annotations is less powerful than using `@PreAuthorize` and `@PostAuthorize`, and the chances that you'll find them in real-world scenarios are small. Even so, I'd like to make you aware of both, but without spending too much time on the details.

You enable the use of these annotations the same way we did for preauthorization and postauthorization by setting to true the attributes of the `@EnableMethodSecurity`. You enable the attributes that represent the use of one kind of annotation, either `@Secure` or `@RolesAllowed`. You can find an example of how to do this in the next code snippet:

```
@EnableMethodSecurity(
  jsr250Enabled = true,
  securedEnabled = true
)
```

Once you've enabled these attributes, you can use the `@RolesAllowed` or `@Secured` annotations to specify which roles or authorities the logged-in user needs to have to call a certain method. The next code snippet shows you how to use the `@RolesAllowed` annotation to specify that only users having the role ADMIN can call the `getName()` method:

```
@Service
public class NameService {

 @RolesAllowed("ADMIN")
 public String getName() {
    return "Fantastico";
 }
}
```

Similarily, you can use the `@Secured` annotation instead of the `@RolesAllowed` annotation:

```
@Service
public class NameService {
 @Secured("ROLE_ADMIN")
 public String getName() {
    return "Fantastico";
 }
}
```

You can now test your example. The next code snippet shows how to do this:

```
curl -u emma:12345 http://localhost:8080/hello
```

The response body is

```
Hello, Fantastico
```

To call the endpoint and authenticating with the user Natalie, use

```
curl -u natalie:12345 http://localhost:8080/hello
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
 "message":"Forbidden",
```

```
  "path":"/hello"
}
```

You can find a full example using the `@RolesAllowed` and `@Secured` annotations in the project ssia-ch9-ex6.

Summary

- Spring Security allows you to apply authorization rules for any application layer, not only at the endpoint level. To do this, we enable the method security functionality.
- The method security functionality is disabled by default. To enable it, we use the `@EnableMethodSecurity` annotation over the configuration class of your application.
- You can apply authorization rules that the application checks before the call to a method. If these authorization rules aren't followed, the framework doesn't allow the method to execute. When we test the authorization rules before the method call, we use preauthorization.
- To implement preauthorization, we use the `@PreAuthorize` annotation with the value of a SpEL expression that defines the authorization rule.
- If we want to only decide after the method call if the caller can use the returned value and if the execution flow can proceed, we use postauthorization.
- To implement postauthorization, we use the `@PostAuthorize` annotation with the value of a SpEL expression that represents the authorization rule.
- When implementing complex authorization logic, you should separate this logic into another class to make your code easier to read. In Spring Security, a common way to do this is by implementing a `PermissionEvaluator`.
- Spring Security offers compatibility with older specifications such as the `@RolesAllowed` and `@Secured` annotations. You can use these, but they are less powerful than `@PreAuthorize` and `@PostAuthorize`, and the chances that you'll find them used with Spring in a real-world scenario are very low.

Previous chapter
< 10 Configuring CORS

Next chapter
12 Implementing filtering at the method level >